

Agilní metodiky

Karel Richta

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2021

Softwarové inženýrství, SWI
05/2021, Lekce 10

<https://moodle.vspj.cz/course/view.php?id=201480>



Agilní metodiky

- Vycházejí z principů „[Manifestu agilních metodik](#)”.
- Psychologie týmu (<http://agilemanifesto.org/>)
- SCRUM (<http://www.scrumalliance.org/>)
- Crystal Clear
(https://en.wikipedia.org/wiki/Crystal_Clear_%28software_development%29)
- XP - Extreme Programming
(<http://www.extremeprogramming.org/>)

Extrémní programování (XP)

Agilní technika vývoje softwaru, která dle „agilního manifestu“ upřednostňuje:

- Týmovou spolupráci a interakci před formálními procesy a nástroji.
- Fungující software před obsáhlou, komplexní dokumentací.
- Spolupráci mezi zákazníkem a vývojáři před specifikacemi, zadáními, dohodnutými kontrakty.
- Rychlou adaptaci na změny v zadání před dodržováním předem stanovených pravidel.

XP: Základní principy

- Řízeno požadavky uživatelů (user stories).
- Vytvírá se v iteracích.
- Pracuje se ve dvojicích.
- Co nejjednodušší návrh (neprogramujeme dopředu), okamžité testování.
- RefaktORIZACE kódu, kód je společný - standardy pro kódování.
- Co nejrychlejší zpětná vazba, uživatel součástí vývojového týmu, minimum dokumentace.
- Nepřehánět zátěž - 40 hodin týdně.

12 technik používaných v XP

- Plánování hrou
- Malé dodávky (releases)
- Metafory
- Jednoduchý návrh
- Párové programování
- Intenzivní testování
- RefaktORIZACE kódu
- Kolektivní vlastnictví kódu
- Spojitá intergace
- 40 hodin/týden
- Zákazník součástí týmu
- Kódové standardy

XP: Plánování hrou

Software se vyvíjí v malých iteracích. Vyvíjí se vždy pouze to, co zákazník v danou chvíli potřebuje.

Iterace se skládá z:

- Sepsání požadavků – user stories (zákazník)
- Rozdělení požadavků na úkoly (vývojáři)
- Časových odhadů jednotlivých úkolů (vývojáři)
- Určení priorit (zákazník)
- Programování (vývojáři)
- Akceptačních testů dle požadavků (zákazník)

XP: Malé dodávky (releases)

Na konci každé iterace (zpracování jednoho příběhu – user story) se vytvoří nová dodávka (release).

To znamená:

- Zákazník dostává poměrně často nové verze (jednou za dva týdny, týden, den).
- Existuje okamžitá zpětná vazba, neboť zákazník může software neustále testovat a připomínkovat.
- Zákazník má k dispozici funkční část softwaru velmi rychle a může ho začít používat.

XP: Metafora

- Jednoduchá paralela nebo příběh, který popisuje budovaný systém slovy a pojmy, které všichni zúčastnění intuitivně chápou.
- Slouží k lepšímu porozumění a komunikaci mezi členy týmu, vývojáři a zákazníky.

XP: Jednoduchý návrh

- Scott W.Ambler:
„Co přesně má software dělat a jak má vypadat víte v nejlepším případě až když ho potřetí přepisujete celý znova.“
- Proto je vhodné udržovat návrh tak jednoduchý, jak to jde. Použít co nejjednodušší, který ještě splní akceptační testy.
- Neprogramovat pro budoucnost, netvořit frameworky, pokud to není nezbytně nutné.

XP: Párové programování

Každá řádka kódu je napsána dvěma programátory u jednoho počítače s jedním monitorem a jednou klávesnicí.

To zajišťuje:

- Vyšší spolehlivost kódu, neboť programátoři se kontrolují navzájem.
- Vyšší čitelnost a pochopitelnost kódu, neboť programátoři se kontrolují navzájem.
- Všeobecné povědomí všech vývojářů o celém kódu.
- Vzájemné učení se od sebe.

XP: Testování

- Neustálé testování je jedna z fundamentálních technik XP, bez testování nemůže XP fungovat.
- Vývojáři píší testy jednotek (unit tests) na každou komponentu vyvíjeného systému, dokonce na každou třídu, tyto testy se spouští neustále.
- Pokud neprochází všechny testy jednotek a akceptační testy pro daný příběh, nelze pokračovat ve vývoji.

XP: RefaktORIZACE

- RefaktORIZACE je technika zlepšování stávajícího kódu beze změny chování.
- Refaktoriše se kvůli:
 - Zjednodušení kódu.
 - Optimalizaci kódu.
 - Lepší čitelnosti a pochopitelnosti kódu.
 - Přidání nové funkcionality.
- Testy jednotek by měly zajistit, že se nezmění stávající chování.

XP: Společné vlastnictví kódu

- Kterýkoliv vývojář v týmu využívajícím XP je oprávněn kdykoliv modifikovat kterýkoliv kus kódu, pokud po modifikaci procházejí testy.

XP: Neustálá integrace

- Veškerý kód je zpřístupněn všem vývojářům ve společném repozitáři.
- Kdykoliv je vyřešen (naprogramován) a otestován nějaký úkol, je kód uložen do repozitáře.

XP: 40 hodin týdně

- Psychicky nebo intelektuálně unavený vývojář je:
 - Nevýkonný
 - Chybující
 - Velmi „naštvaný“
- Proto je nutné vývojovému týmu zajistit klid na práci, důstojné pracovní prostředí a hlavně je nepřetěžovat.

XP: Zákazník v týmu

- Součástí týmu by měl být „on-site customer“, který představuje typického uživatele a je kdykoliv k dispozici vývojářům aby:
 - Zodpovídal otázky vývojářů během práce
 - Tvořil akceptační testy pro jednotlivé příběhy
 - Testoval a podával zpětnou vazbu

XP: Kódové standardy

- Je nutné, aby všichni programátoři dodržovali stejné standardy:
 - Stejně zarovnání kódu
 - Stejně konvence pro identifikátory
 - Stejně nastavené vývojové prostředí

Kdy se hodí XP?

- Když je limitovaný čas nebo finance.
- Když zákazník není přesně co chce, nebo není schopen dodat dostatečně přesné zadání.
- Když zákazník chce spolupracovat na projektu.
- Když se ví, že se bude systém měnit.
- Když jsou členové týmu schopni spolupracovat a dobře spolu vycházejí.

Kdy se nehodí XP?

- Když je nutný větší tým vývojářů (více než 15).
- Když je vývojový cyklus (oprava, překlad, sestavení spuštění) příliš dlouhý (5 minut, hodina).
- Když jsou testy příliš dlouhé (minuty, hodiny).
- Když nelze zákazníka dostat do týmu.
- Když zákazník nechápe principy XP.

Srovnání

- XP představuje alternativu k UP, či vodopádovému modelu.
- Není to nekoordinované „hackování“ kódu zdivočelými programátory.
- Aby fungovalo, musí mít tým jisté zkušenosti a musí dodržovat všech 12 technik.
- XP není všelék, hodí se jen někdy, ale někdy může přinášet lepší výsledky za kratší čas.

SCRUM (pojem z rugby)

- Žádné týmové hierarchie, tým do 7 osob, pracuje se v jedné místnosti (osmóza), tým si volí šéfa (scrum master).
- Krátké iterace (sprints).
- Plánovací schůzka před každou iterací.
- Seznam akcí pro danou iteraci (backlog).
- Denní skrumáže (co jsem udělal, co budu dělat, jaké mám problémy).
- Retrospekce po každé iteraci (heartbeat session).
- Vlastníkem kódu je tým - standardy pro kódování.

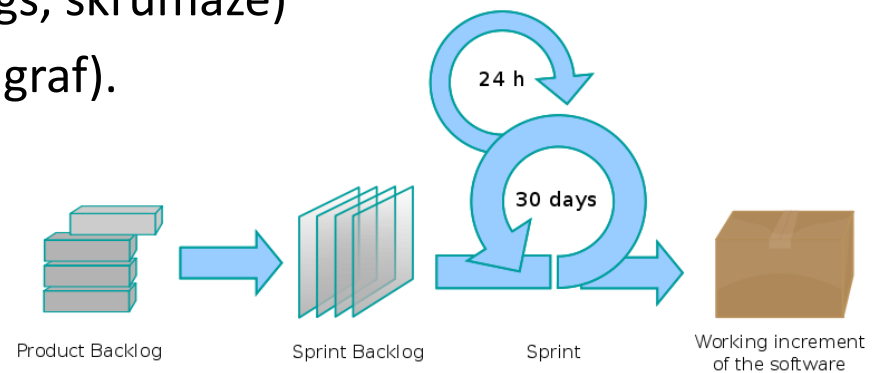
Role v metodice SCRUM

Metodika SCRUM stanovuje následující role:

- vedoucí projektu (Scrum Master) – řídí procesy a celkovou práci,
- zadavatel (Product Owner) – který reprezentuje zainteresované osoby (stakeholders) a
- vývojář (TeamMember) – který zahrnuje členy týmu.

SCRUM - postup

- Vytvoření plánu projektu (project backlog)
 - Sestavení případů užití (user stories)
 - Definice potřebných úloh (sprintů).
- Realizace plánu projektu
 - Výběr sprintu (sprint backlog)
 - Realizace sprintu
 - Denní setkání (SCRUM Meetings, skrumáže)
 - Sledování postupu (burndown graf).



Zdroj: Wikipedia

SCRUM – Sprints

- Rozdělit práci na menší celky – „sprinty” (iterace).
- Během sprintu by se měla stihnout celá úloha – analýza, návrh a implementace, včetně testování a revizí.
- Doba trvání jednoho sprintu cca do 30 dnů (rozhoduje si to tým).
- Během sprintu se plán sprintu nemůže měnit (je zmrazen). Po ukončení sprintu tým prezentuje vytvořený přírůstek a dokumentuje jeho funkčnost.
- Výsledkem je přírůstek použitelného kódu.

SCRUM: Výběr sprintu

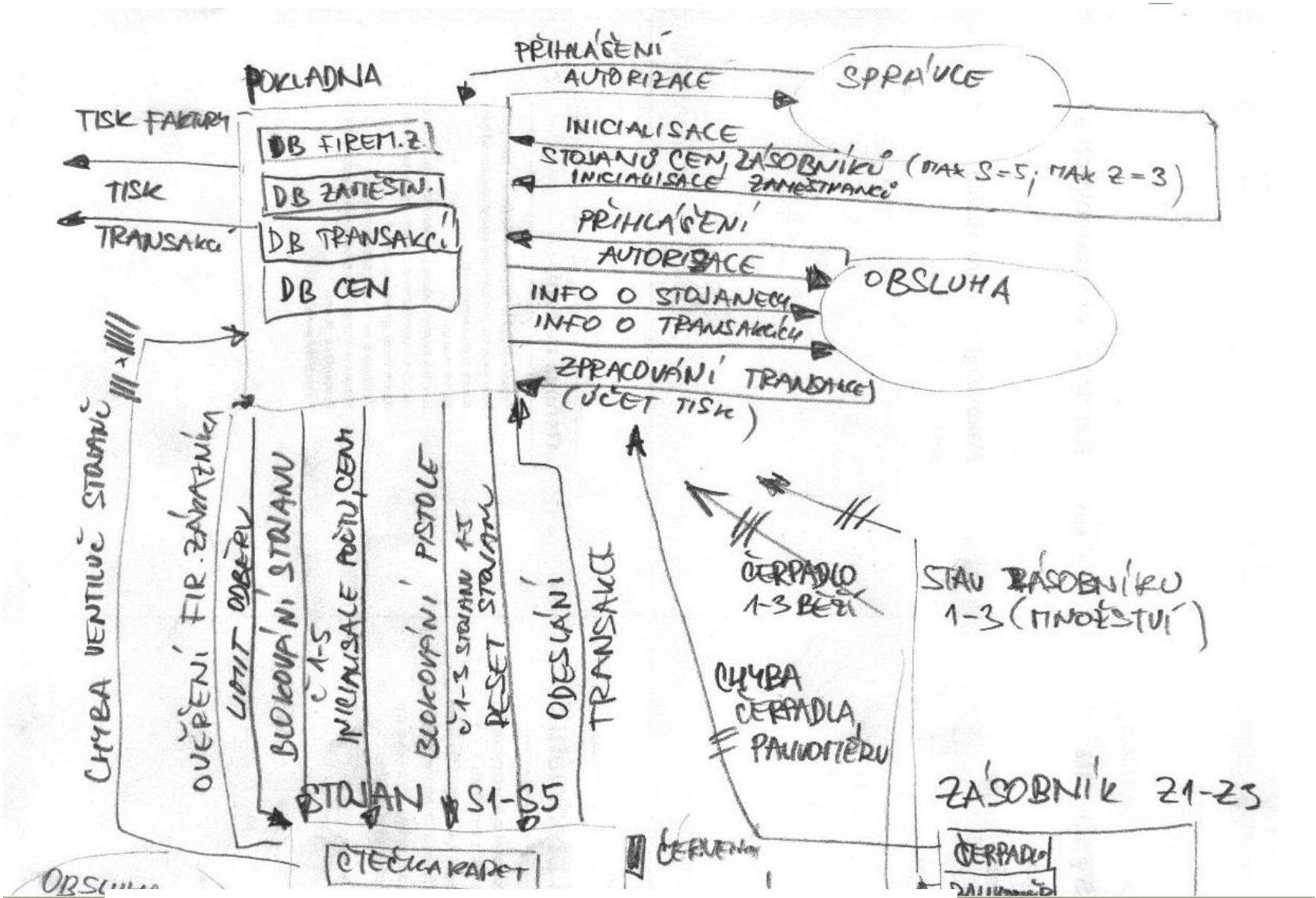
- Zadání je vybráno z katalogu požadavků (product backlog) na schůzce (sprint planning meeting), kde zadavatel (product owner) informuje o požadavcích, které je zapotřebí realizovat – jsou součástí katalogu požadavků (product backlog).
- Ohodnocení úloh body (bezrozměrnými, jen poměr, pro první plán: 1bod = 1man-day).
- Výběr pořadí sprintů.
- Tým si vybere co je schopen realizovat v rámci jednoho sprintu a určuje si plán sprintu (sprint backlog) - hierarchickou sadu požadavků, které by měl sprint realizovat.

Výhody metodiky SCRUM

- Metodika Scrum umožňuje vytváření samoorganizujících týmů – pro členy týmu to znamená větší pocit svobody, ale také větší zodpovědnost.
- Klíčový princip metodiky Scrum je připravenost na změny (nazývané requirements churn) a pochopení, že tuto skutečnost nelze řešit klasickou organizací a plánováním.

Agilní přístupy a dokumentace

- Představa, že v agilních metodikách se nevytváří žádná dokumentace, či modely, ale pouze kód, není pravdivá. V agilních metodikách modelujeme, dokumentujeme, ale pouze v jiné formě a pouze to, co je nezbytně nutné.
- Uživatelské příběhy (user stories) popisují funkčnost a detaily jsou zachyceny v testech.
- Další dokumentací je samozřejmě dobře strukturovaný a komentovaný zdrojový kód.
- Modely také existují ale například pouze jako náčrty na tabuli či na papíře. Jelikož jsou běžné krátké obrázky, kód se často mění (refaktoruje), stálo by přepracování modelů či jiných dokumentů spoustu úsilí nebo by se tyto staly brzy neaktuální, proto opravdu dokumentujeme jen důležité věci.



Podzimní volání formálních funkcí (rychlakvaš)

Agilní přístupy a architektura

- Není pravda, že v agilních metodikách nenavrhujeme žádnou architekturu – je pravdou, že na začátku projektu si nesesedneme ke všem požadavkům (protože je ani nemáme) a neřešíme „papírovou architekturu“.
- V úvodních iteracích ale řešíme určitou kostru aplikace, vrstvy, způsob ukládání dat, komunikaci s ohledem na celé řešení, definujeme rozhraní vrstev, jelikož toto mohou být zásadní technické problémy (např. integrace se zastaralým – „legacy“ - bankovním systémem, pravidelné dávkové importy, implementace standardů), které mohou silně ovlivnit celé řešení – použitou technologii, způsob komunikace mezi vrstvami, nutnost prototypů apod.
- Detailní architekturou se pak zabýváme vždy jen pro danou iteraci (tj. jen pro scénáře které nyní implementujeme), neřešíme architekturu s ohledem na požadavky, které bychom měli implementovat v budoucích iteracích, protože se mohou změnit priority, tyto požadavky se mohou posunout do dalších verzí, či úplně zrušit a naše snaha může přijít vniveč. Neděláme tedy architekturu „do šuplíku“, ale jen pro aktuální potřebu.

Agilní jsou/mají být jen vývojáři?

- Agilní nemohou být jen vývojáři, obchodníci a management musí agilnímu přístupu rozumět také.
- Pokud zákazníkovi řekneme (tedy spíše naši obchodníci), že jsme schopni dodat opravdu všechno, co si vymyslel, za tu cenu, v tom čase a v té kvalitě (což se opravdu často děje), bez ohledu na to, co říká trojúhelník kvality, pak nás samozřejmě ani agilní přístupy nezachrání.
- Pokud zákazník nebude chtít spolupracovat, nebude se chtít účastnit pravidelných schůzek (user play), kde si hraje s dosud vyvinutým řešením, komentuje se a připomínkuje, nelze očekávat úspěch.
- Musíme zákazníka učit, vysvětlovat, proč je nutné vidět aplikaci a korigovat své i naše představy, že jen tak dostane opravdu to, co očekával.
- Je to těžké, protože zákazník byl po léta zvyklý na začátku nadiktovat všechny, tedy i ty nepotřebné požadavky (kterých je podle Standish Group více než polovina) a na konci očekával řešení.

Podpora vedení firmy není třeba?

- Podpora vedení firmy je velmi důležitá. Nestačí, aby vedení vývojářům řeklo: „ano, nějak si to teda udělejte (rozuměj zaveď agilní metodiku)“ – to je další častou chybou a naivní představou.
- Co je ještě horší, je zavádět agilní praktiky a životní cyklus potají. Tajné zavádění či nedostatečná podpora ze strany managementu v podstatě úplně zhatí celou snahu, jelikož bez podpory managementu (který má tuto vizi protlačit) a bez oficiálních prostředků a schválení (vývojáři absolvují tréninky, zpomalí se tempo vývoje) agilní přístup ani zavést nelze, jedná se totiž o úplnou změnu chování a myšlení všech lidí v organizaci a také o změnu vystupování směrem k zákazníkovi (což potají moc dobře nejde :-).
- Standish Group řadí podporu managementu (executive support) na 1.místo svého pravidleného tzv. „Chaos report“, který popisuje situaci ohledně projektů v IT. Angažovanost uživatelů v projektu řadí až na 2.místo.

Agilní přístupy obecné shrnutí

- Agilní přístupy jako Scrum, XP, Lean development, OpenUP, AUP apod. většinou představují sadu doporučení, co se v praxi osvědčilo.
- Metodika přesně říká co, kdy, kdo a jak má dělat. Zde se často popisují kroky, které je možné provádět při vývoji software, jaké artefakty mohou zachycovat různé informace, prostě jaké činnosti se v praxi osvědčily.
- Na nás pak je, abychom si vybrali, co konkrétně nyní potřebujeme, uznáme za vhodné dělat, tj. vytvořili si vlastní proces.

Nutné je přizpůsobení metodiky

- Je to pochopitelné, každá organizace je jiná, má jinou strukturu, kulturu, distribuce týmu se různí, členové týmu mají jiné zkušenosti a znalosti, jejich povahové rysy jsou různé, také zákazník se chová jinak.
- Proto není možné postupovat vždy podle stejné či podobné metodiky, ale je třeba si daný proces vývoje upravit podle potřeb daného projektu (přesně toto doporučují či říkají dané přístupy).
- Samozřejmě **je třeba dodržet a následovat určité zásady a principy**.
- Funguje to stejně jako s bufetem ve formě švédských stolů. Bereme si jen to, co máme rádi (ne všechno – častá dezinterpretace UP, že je příliš těžkopádná; nikde však není psáno, že máme dělat vše, co UP popisuje), s čím máme dobré zkušenosti, sem tam zkusíme něco nového a snažíme se tomu přijít na chuť, ale musíme dodržovat určité zásady. Jídlo dáváme na talíř, většinou jíme příbory, nápoje naléváme do skleniček apod.

Zdroj: Procházka: [Problémy s adopcí agile přístupů](#)

The End